# Программирование на Python

## Лекция 6

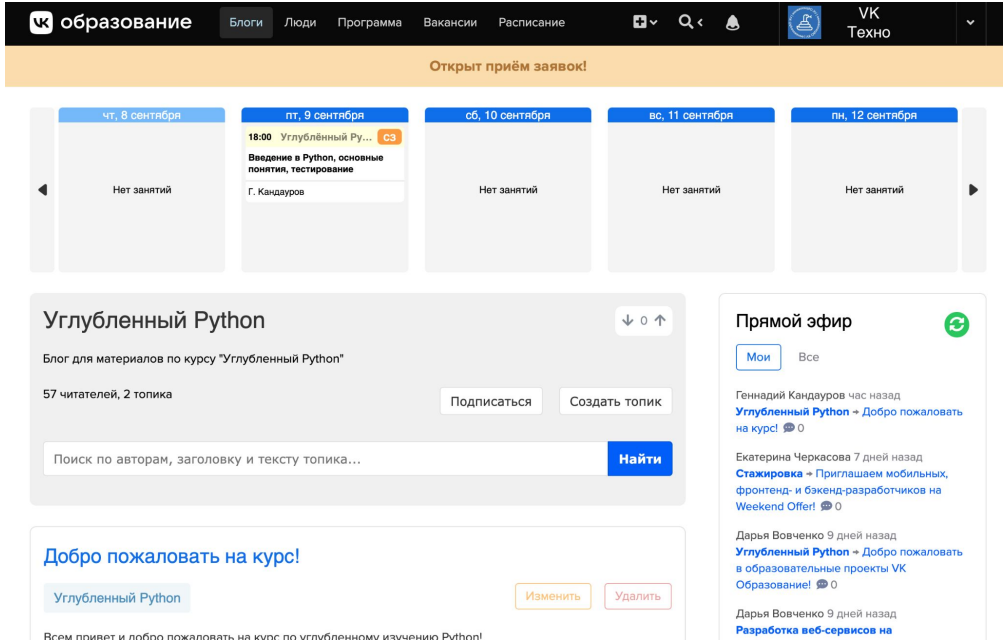
## Классы, введение в ООП
## Итераторы и генераторы

Кандауров Геннадий

education

# Напоминание отметиться на портале

+ оставить отзыв



# Квиз про прошлой лекции

# Содержание занятия

1. Классы, ООП
2. Объектная модель
3. Итераторы и генераторы

# Классы

# Классы

Шаблон, по которому создаются экземпляры (объекты).

Класс определяет, какие атрибуты и методы будут у экземпляров данного типа.

- Организация и структурирование кода
- Повторное использование
- Инкапсуляция
- Наследование
- Полиморфизм

# Классы

**object.__new__(cls[, ...])**

Статический метод, создает новый экземпляр класса.

После создание экземпляра вызывается (уже у экземпляра) метод `__init__`.

`__init__` ничего не должен возвращать (кроме None), иначе - `TypeError`

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls, *args, **kwargs)
        return cls._instance
```

# Классы

**object.__del__(self)**

Финализатор

```
class Connector:
    def __init__(self, db_name):
        self.conn = DbDriver(db_name)
    def __del__(self):
        self.conn.close()
        print("DEL")


db = Connector("users")
del db  # ???
```

# Классы: доступ к атрибутам

Чтобы найти атрибут объекта `obj`, python обыскивает:

1. Сам объект (`obj.__dict__` и его системные атрибуты)
2. Класс объекта (`obj.__class__.__dict__`).
3. Классы, от которых унаследован класс объекта (`obj.__class__.__mro__`)

# Классы: наследование

```
class Timing:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def duration(self):
        print("Timing.duration")
        return self.end - self.start


class MinuteTiming(Timing):
    def duration(self):
        print("MinuteTiming.duration")
        seconds = super().duration()
        return seconds / 60
```

```
>>> m = MinuteTiming(1000, 7000)
>>> m.duration()
MinuteTiming.duration
Timing.duration
100.0
```

# Классы: super

**super**()

**super**(type, object_or_type=None)

Возвращает прокси-объект, который делегирует вызовы методов родительскому или родственному классу type.

Полезно для доступа к унаследованным методам, которые были переопределены в классе.

- mro - линейный порядок поиска атрибутов
- C3-линеаризация - алгоритм вычисления mro

# Классы: магические атрибуты

**Классы**

`__name__` — имя класса

`__module__` — модуль, в котором объявлен класс

`__qualname__` — fully qualified имя

`__doc__` — докстринг

`__annotations__` — аннотации статических полей класса

`__dict__` — namespace класса

**Методы**

`__self__` — объект класса

`__func__` — сама функция, которую мы в классе объявили

# Классы: магические атрибуты

**Поля, относящиеся к наследованию**

`__bases__` — базовые классы

`__base__` — базовый класс, который указан первым по порядку

`__mro__` — список классов, упорядоченный по вызову функции `super`

```
class B(A): pass
>>> B.__bases__
(__main__.A,)
>>> B.__base__
__main__.A
>>> B.__mro__
(__main__.B, __main__.A, object)
```

# Классы: магические методы

**Доступ к атрибутам**

- `__getattribute__(self, name)`
- `__getattr__(self, name)`
- `__setattr__(self, name, val)`
- `__delattr__(self, name)`
- `__dir__(self)`

# Классы: магические методы

**object.__call__(self[, args...])**

```
class Adder:
    def __init__(self, val):
        self.val = val

    def __call__(self, value):
        return self.val + value

a = Adder(10)
a(5)  # 15
```

# Классы: магические методы

**`object.__hash__(self)`**

Вызывается функцией `hash()` и коллекциями, которые построены на основе hash-таблиц.

Нужно, чтобы у равных объектов был одинаковый hash.

Если определен метод `__eq__` и не определен `__hash__`, то объект не может быть ключом в hashable коллекции.

```
>>> key1 = (1, 2, 3)
>>> key2 = (1, 2, 3, [4, 5])
>>> s = set()
>>> s.add(key1)  # ???
>>> s.add(key2)  # ???
```

# Классы: магические методы

**`object.__slots__`**

Позволяет явно указать поля, которые будут в классе.

В случае указания `__slots__` пропадают поля `__dict__` и `__weakref__`.

Используя `__slots__` можно экономить на памяти и времени доступа к атрибутам объекта.

```
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

# Классы: __init_subclass__

```
class Timing:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    @classmethod
    def __init_subclass__(cls, **kwargs):
        print("INIT subclass", cls, kwargs)


class MinuteTiming(Timing):
    def duration(self):
        print("MinuteTiming.duration")
        seconds = super().duration()
        return seconds / 60
```

# Итераторы

**Итератор** представляет собой объект-перечислитель, который для данного объекта выдает следующий элемент, либо вызывает исключение, если элементов больше нет.

```
num_list = [1, 2, 3]
itr = iter(num_list)
print(next(itr))  # 1
print(next(itr))  # 2
print(next(itr))  # 3
print(next(itr))  # StopIteration
```

```
class SpecialIterator:
    def __init__(self, limit):
        self.limit = limit
        self.counter = 0
    def __iter__(self): return self
    def __next__(self):
        if self.counter < self.limit:
            self.counter += 1
            return self.counter
        else:
            raise StopIteration

iter_obj = SpecialIterator(3)
print(next(iter_obj))
```

# Генераторы

**Генератор** – подпрограмма, которая может возвращать очередное значение и автоматически сохранять и возобновлять своё состояние для возврата следующего значения;

объект, который при вызове `next()` возвращает следующий элемент по алгоритму его работы.

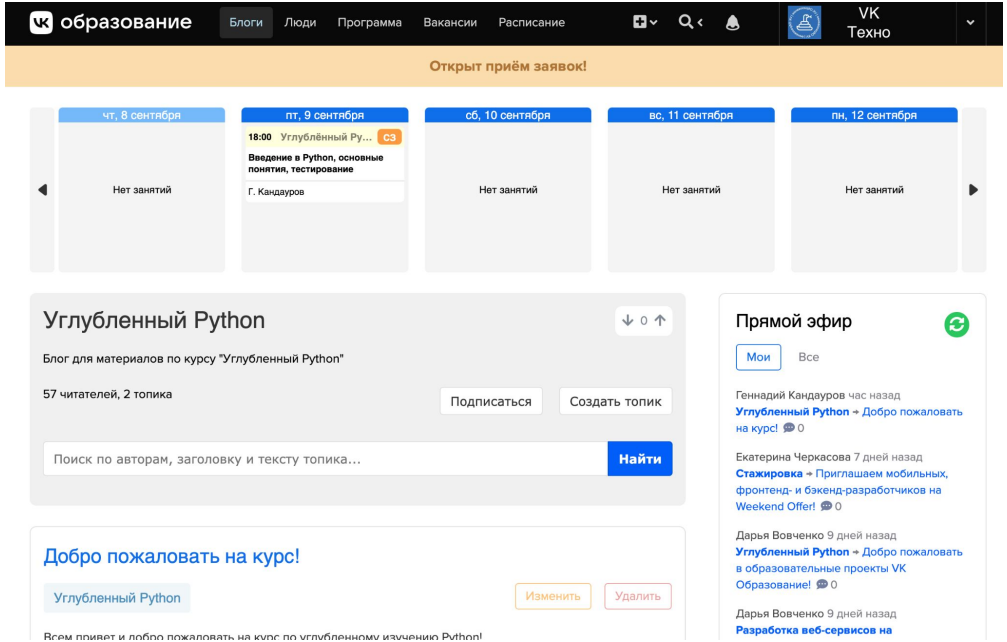
```
def gen(count):
    while count > 0:
        yield count
        count -= 1
    return count  # будет аргументом StopIteration

for i in gen(5):
    print(i)  # 5, 4, 3, 2, 1
```

# Домашнее задание #06

- Генератор

# Напоминание отметиться на портале Vol 2

+ оставить отзыв после лекции



# Спасибо за внимание

education